INSTITUTE OF ENGINEERING & MANAGEMENT, KOLKATA

COURSE TITLE: BACHELOR OF TECHNOLOGY

SUBJECT: PYTHON PROGRAMMING LABORATORY

PROJECT TITLE: SPORTS PERFORMANCE STATISTICS EXPLORER

NAME: 1. Debanik Dey 2. Avasyu Roy

3. Debarghya Bose 4. Sneha Pramanik 5. Debadrita Samadder

ROLL NO: 291, 294, 292, 295, 301

ENROLLMENT NO: 12025052003312, 12025052003315, 12025052003313, 12025052003316, 12025052003322

SECTION: D

SUBMITTED TO: Prof. Indranil Basu

DEPARTMENT NAME: ELECTRONICS AND COMMUNICATION ENGINEERING

ACADEMIC SESSION: 2026-2027

DATE OF SUBMISSION: 11/11/2026

Abstract

This project develops a Python-based web application, built with Flask, to analyse event times and scores of anonymised athletes across repeated training sessions. It validates and ranks results with dedicated functions, stores athlete and session records in CSV files, models each athlete with an *Athlete* class, computes the mean, variability and improvement with NumPy, produces ranked event reports with Pandas, and draws comparative bar charts and time-series plots with Matplotlib.

The central analytical question is whether an athlete's improvement is consistent across sessions or driven by a single result. Five pieces of evidence are combined: a least-squares trend, the share of session-to-session gains, Spearman's rank correlation, the share of all gains supplied by the largest single jump, and a leave-one-out trend. On the included synthetic dataset (8 athletes, 3 events, 190 results), 8 of 24 athlete-event series show consistent improvement, while 10 are classified as driven by one result. Notably, the athlete ranked second in the 100 m on best time owes that rank entirely to the final session; removing it eliminates the upward trend. The results show why a best-performance ranking should be reported together with a consistency check.

Keywords

Sports performance; athlete analytics; Python; Flask; CSV; NumPy; Pandas; Matplotlib; ranking; consistency analysis; leave-one-out; Spearman correlation.

Table of Contents

1. Introduction
2. Problem Statement and Objectives
3. Software and System Requirements
4. System Design and Methodology
5. Implementation of Major Modules
6. Results and Discussion
7. Testing and Validation
8. Limitations and Future Scope
9. Conclusion
10. References
11. Appendix: CSV Format and Execution

1. Introduction

Coaches routinely record sprint times, run times and jump distances for each athlete at every training session. Kept in a spreadsheet, these records make it difficult to compare athletes fairly, to see how variable each athlete is, and above all to judge whether apparent progress is genuine. A single exceptional day can make an athlete look much improved when the rest of their sessions show no change.

The developed application is a Flask web site that runs in any browser. It displays summary indicators, a ranked event report with an automated assessment for every athlete, comparative bar charts, time-series plots, squad summaries, a session grid and a per-athlete trend view. Users can work with the preloaded synthetic dataset, log new results through validated web forms, or import and export anonymised CSV files.

2. Problem Statement and Objectives

Problem statement: Analyse event times or scores for anonymised athletes across sessions. Validate and rank scores with functions; manage athlete/session records in files; implement an Athlete class; use NumPy for mean, variability and improvement; Pandas for event reports; draw comparative bars and time-series plots. Assess whether improvement is consistent across sessions rather than driven by one result.

The principal objectives are:

- Validate athlete IDs, events, session numbers, dates and results against plausible event ranges.
- Rank athletes per event with competition ranking that respects the direction of each event.
- Read, append, delete, import and export athlete and result records in CSV files.
- Apply an Athlete class to encapsulate each athlete’s results and statistical methods.
- Compute mean, standard deviation, coefficient of variation and improvement with NumPy.
- Generate ranked event reports, session grids and squad summaries with Pandas.
- Draw comparative bar charts and time-series plots, and classify each athlete’s improvement as consistent or driven by one result, with written reasons.

3. Software and System Requirements

Component	Purpose
Python 3	Core program execution
Flask (with Jinja2 templates)	Web server, routing, forms and HTML page rendering
csv, re, datetime, math	CSV file handling, ID pattern checks, date and numeric validation
NumPy	Arrays, mean, standard deviation, differences, trend fitting and correlation
Pandas	Event reports, pivot tables, group-by squad summaries and CSV export
Matplotlib (Agg backend)	Comparative bar charts and time-series plots rendered as images
HTML / CSS	Responsive user interface for desktop and mobile
Web browser	Open the application at http://127.0.0.1:5000

The program runs on any computer with Python 3 and the four third-party packages listed in *requirements.txt*. No database is required: all records are kept in two CSV files in the *data* folder.

4. System Design and Methodology

4.1 Input data and schema

Two files are used. *athletes.csv* holds *athlete_id* (an anonymised identifier such as A07) and *squad* (Junior or Senior). *results.csv* holds *athlete_id*, *event*, *session* (a whole number from 1), *date* (YYYY-MM-DD) and *result*. Each event defines its unit, whether higher or lower is better, and a plausible range:

Event	Unit	Better result	Accepted range
100 m sprint (100m)	seconds	Lower	9.5 – 20.0 s
400 m run (400m)	seconds	Lower	43.0 – 90.0 s
Long jump (long_jump)	metres	Higher	2.0 – 9.0 m

4.2 Data preparation

The script *seed_data.py* creates a reproducible demonstration dataset with a fixed random seed (42). Eight athletes complete eight weekly sessions in each of the three events, giving 192 possible results; two sessions are deliberately missed, so 190 results are stored. Each athlete follows a designed pattern so the analysis has something to distinguish: steady improvers (A01, A02, A07), one-off good days (A03 in session 6, A08 in session 8), a breakthrough that holds (A06 from session 4), a flat athlete (A04) and a declining athlete (A05). Every row read from file is validated again; invalid rows are skipped.

4.3 Analysis workflow

1. Read athletes.csv and results.csv and validate each row.
2. Build an Athlete object per athlete and attach its results by event and session.
3. Compute per-athlete statistics with NumPy (mean, SD, CV, best, latest, improvement).
4. Run the consistency analysis and assign a verdict with written reasons.
5. Rank athletes by best result and assemble the Pandas event report and squad summary.
6. Render the bar chart, time-series plot and per-athlete trend charts with Matplotlib.
7. Display everything in the web dashboard, with CSV export of the report.

4.4 Statistical measures

The mean describes typical performance and the sample standard deviation (ddof = 1) describes spread; the coefficient of variation CV = SD ÷ mean × 100 allows variability to be compared between events. Improvement is the change from the first to the latest session, signed so that a positive value always means better (a lower time or a longer jump). For the consistency analysis, times are multiplied by −1 so that higher is better for every event, and the following are computed with at least four sessions:

- **Trend slope** – least-squares line through all sessions using *np.polyfit*.
- **Gains hit rate** – share of session-to-session changes (*np.diff*) that were improvements.
- **Rank correlation** – Spearman’s ρ between session number and performance.
- **Biggest-jump share** – share of all gains contributed by the largest single jump, and how many times larger that jump is than the median change (its *standout* ratio).
- **Leave-one-out trend** – the slope recomputed with each session removed; the weakest of these, as a fraction of the full slope, shows whether one session is carrying the trend.

Verdict	Rule applied
Consistent improvement	Positive trend, hit rate ≥ 60%, ρ ≥ 0.7, trend at least twice the residual noise, and no single session propping up the trend.
Driven by one result	Positive trend, but the biggest jump is ≥ 2.5× a typical change and either supplies ≥ 60% of all gains or its removal halves the trend.
Inconsistent improvement	Positive trend that is above noise but too irregular to be called steady.
No clear improvement	Flat trend, or an upward drift smaller than 1.5× the normal session noise.
Declining	Trend in the wrong direction with ρ ≤ −0.5.

5. Implementation of Major Modules

Module / function	Functionality
validate_score(), validate_record(), validate_athlete_id()	Numeric, range, date, ID and duplicate checks with readable error messages
rank_scores()	Direction-aware competition ranking (1, 2, 2, 4)
Athlete, add_result(), history()	Object-oriented athlete records stored by event and session
mean(), std(), cv(), best(), improvement(), session_deltas()	NumPy statistics for each athlete and event
load_athletes(), add_result(), delete_result(), import_results()	CSV file reading, appending, rewriting and validated import
consistency_analysis(), spearman()	Trend, hit rate, dominance, leave-one-out and verdict
event_report(), session_pivot(), squad_summary()	Pandas DataFrames for reports and summaries
comparative_bar(), cohort_timeseries(), athlete_trend()	Matplotlib charts encoded as images for the web pages
dashboard(), athlete_page(), records(), export_report()	Flask routes for the pages, forms and CSV downloads

5.1 Interface layout

A top navigation bar links three pages. The **Dashboard** has event buttons (100 m, 400 m, long jump) and a squad filter, four indicator cards (athletes, top-ranked athlete, cohort mean, consistent improvers), a verdict bar, the ranked event report with CSV download, the comparative bar chart, the time-series plot, a squad summary and a session grid. Each **Athlete** page shows, per event, a trend chart, statistics, the reasons for the verdict and a session table with delete controls. The **Records** page provides forms to log a result, register an athlete, and import or export CSV files. A **Method** page explains the assessment rules.

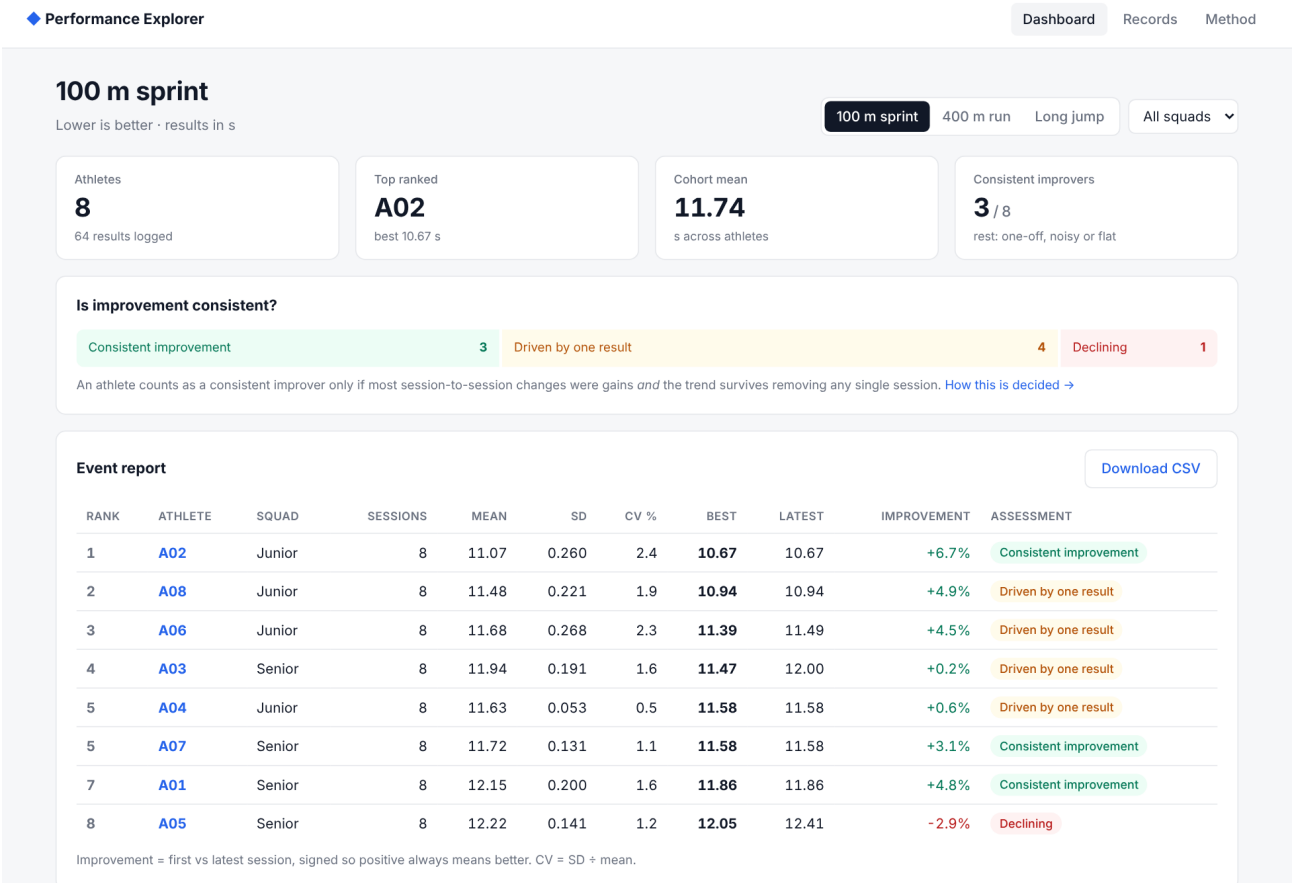


Figure 1. Dashboard of the web application for the 100 m sprint.

5.2 Ranking rule

Athletes are ranked on their best result for the selected event. For the 100 m and 400 m the lowest time ranks first; for the long jump the longest distance ranks first. Equal results share a rank and the next rank is skipped (standard competition ranking), for example 10.5 s, 11.0 s and 11.0 s receive ranks 1, 2 and 2. Results outside an event's accepted range are rejected before ranking.

6. Results and Discussion

6.1 Demonstration dataset and descriptive findings

The following results were computed from the fixed-seed synthetic dataset included with the application. They will differ if another CSV is imported or new results are logged.

Indicator	Observed value
Athletes	8 (4 Junior, 4 Senior)
Events	3 (100 m, 400 m, long jump)
Possible / stored results	192 / 190 (2 missed sessions)
Athlete-event series analysed	24
Consistent improvement	8 of 24 (33.3%)
Driven by one result	10 of 24 (41.7%)
Inconsistent improvement	1 of 24
No clear improvement	2 of 24
Declining	3 of 24

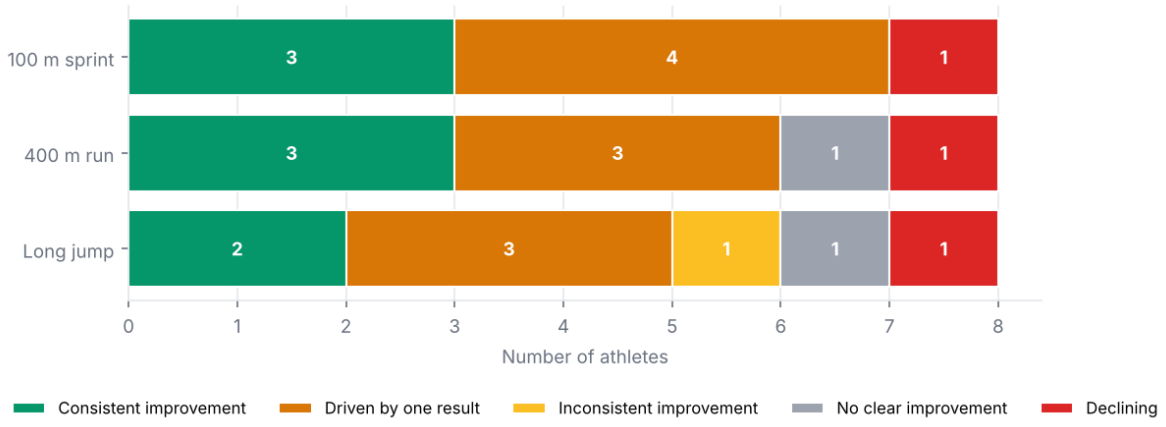


Figure 2. Consistency verdicts for every athlete, grouped by event.

6.2 Event report: 100 m sprint

Rank	Athlete	Squad	Mean (s)	SD (s)	CV %	Best (s)	Change	Assessment
1	A02	Junior	11.07	0.260	2.4	10.67	+6.7%	Consistent improvement
2	A08	Junior	11.48	0.221	1.9	10.94	+4.9%	Driven by one result
3	A06	Junior	11.68	0.268	2.3	11.39	+4.5%	Driven by one result
4	A03	Senior	11.94	0.191	1.6	11.47	+0.2%	Driven by one result
5	A04	Junior	11.63	0.053	0.5	11.58	+0.6%	Driven by one result
5	A07	Senior	11.72	0.131	1.1	11.58	+3.1%	Consistent improvement
7	A01	Senior	12.15	0.200	1.6	11.86	+4.8%	Consistent improvement
8	A05	Senior	12.22	0.141	1.2	12.05	−2.9%	Declining

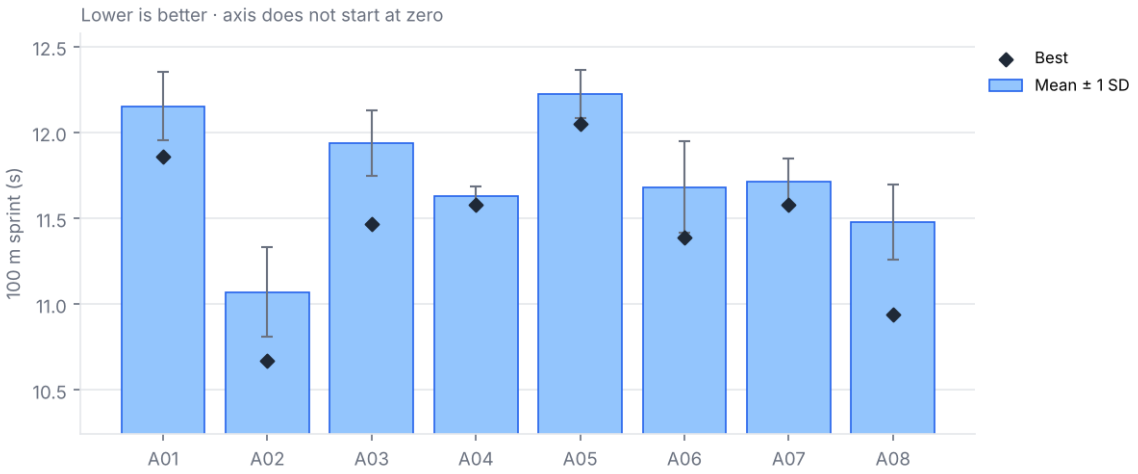


Figure 3. Mean 100 m time (± 1 SD) and best time for each athlete.

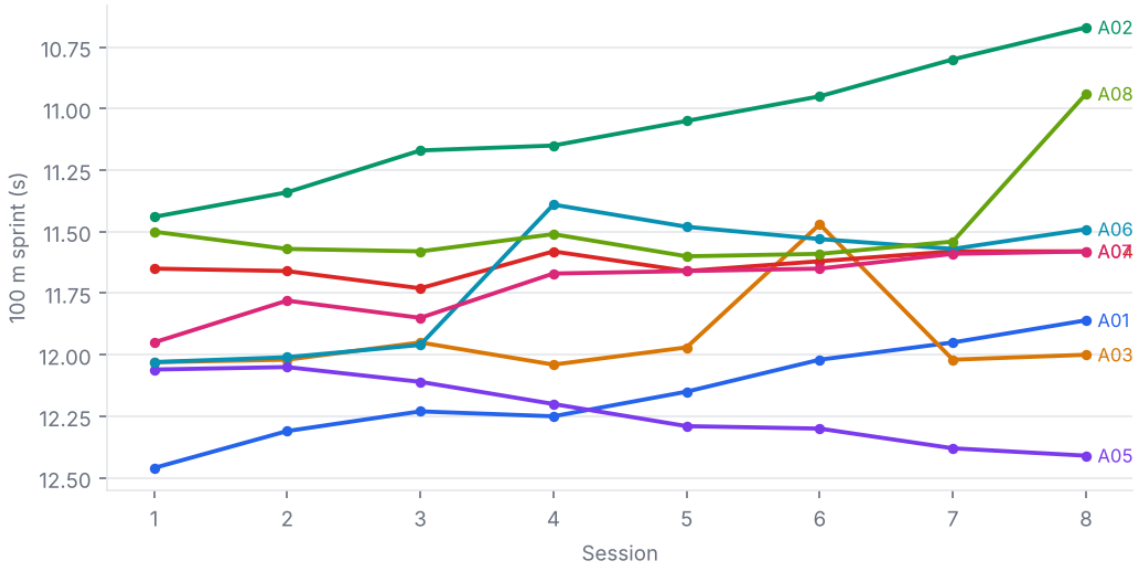


Figure 4. 100 m results across the eight sessions for all athletes.

A02 is ranked first on best time and improved in every one of seven session-to-session changes. A08 is ranked second, but Figure 4 shows that A08 was flat for seven sessions and then dropped 0.60 s in session 8. A04 and A07 tie for fifth on an identical best of 11.58 s and share the rank. In the squad summary, Juniors improved by 4.17% on average against 1.31% for Seniors, yet only one of the four Juniors shows consistent improvement; most of the Junior average comes from one-result athletes.

Squad	Athletes	Mean result (s)	Avg improvement	Consistent improvers
Junior	4	11.47	+4.17%	1
Senior	4	12.01	+1.31%	2

6.3 Consistent improvement versus one result

Measure (100 m)	A02	A08	A06	A03
First-to-latest improvement	+6.7%	+4.9%	+4.5%	+0.2%
Gains hit rate	100%	57%	57%	71%
Spearman ρ	+1.00	+0.05	+0.60	+0.35
Biggest-jump share of gains	22%	82% (S7→8)	79% (S3→4)	75% (S5→6)
Biggest jump vs typical change	1.7×	10.0×	11.4×	7.1×
Trend kept without weakest session	96%	0%	89%	7%
Verdict	Consistent	One result	One result	One result

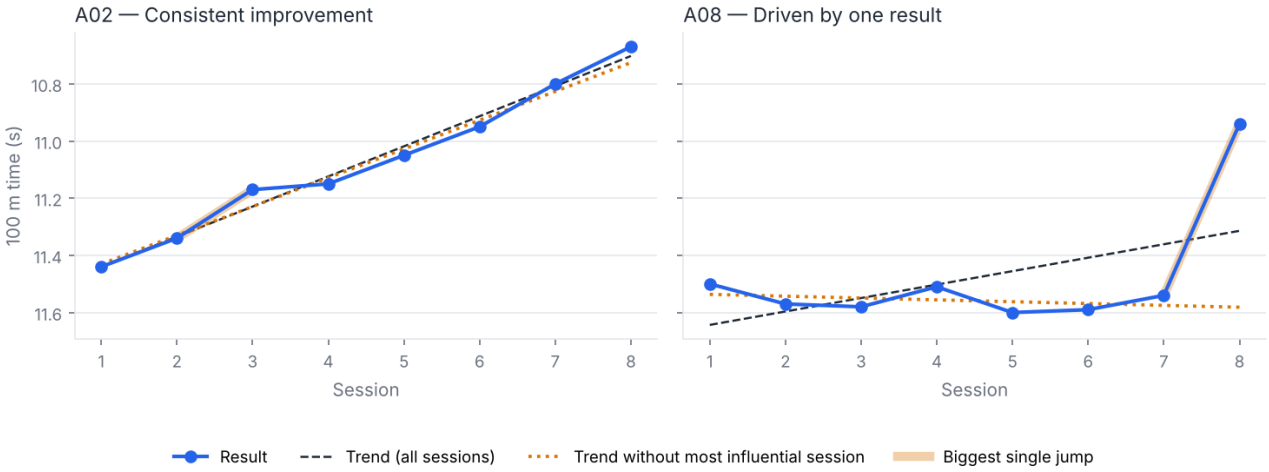


Figure 5. Trend analysis for A02 (left, consistent) and A08 (right, driven by session 8). Dotted line: trend without the most influential session.

A02 and A08 both appear improved on a simple first-versus-latest comparison (6.7% and 4.9%). The consistency analysis separates them clearly: removing any single session leaves 96% of A02's trend, whereas removing session 8 leaves none of A08's. A08's ranking and improvement therefore rest on one result and should be confirmed in later sessions before being treated as real progress.

6.4 Interpretation

"Driven by one result" covers two different situations. A03 produced one fast run in session 6 and then returned to previous levels (latest 12.00 s), which is a one-off. A06 improved sharply at session 4 and held the new level, so 89% of the trend survives removing any single session; this is a breakthrough rather than a fluke, and the per-athlete chart makes the difference visible. Across events, A01 and A02 are consistent in all three, A07 in two of three, and A05 declines in all three, showing that the classification is stable for athletes with clear patterns.

The rules are deliberately strict. A04, designed as a flat athlete, is labelled "driven by one result" in the 100 m because a single 0.15 s improvement accounts for most of a tiny 0.6% net gain. This shows that verdicts should be read together with the size of the improvement. Moreover, the built-in data are generated by a mathematical procedure, so the numbers demonstrate the behaviour of the application rather than evidence about real athletes, and the analysis describes patterns without explaining their causes.

7. Testing and Validation

Test case	Expected behaviour	Observed
Result "abc" for 100 m	Reject non-numeric result	Rejected: "Result must be a number."
100 m result of 4.2 s	Reject value outside 9.5–20.0 s	Rejected with range message
Result "nan"	Reject non-finite value	Rejected
Athlete ID "John Smith"	Reject names; anonymised IDs only	Rejected
Duplicate athlete A01	Reject existing ID	Rejected
Second result for A01, 100 m, session 3	Reject duplicate session	Rejected
Date 2026/10/05	Require YYYY-MM-DD	Rejected
Unknown event "shotput"	Reject event not configured	Rejected
CSV import of 6 rows (4 invalid)	Append valid rows, report bad ones by line	2 imported; 4 rejected with reasons
CSV with wrong headers	Reject file and list missing columns	Rejected; 5 columns listed
Ranking 11.0, 10.5, 11.0 s	Ranks 1, 2, 2 (lowest time first)	1, 2, 2
Delete a result	Rewrite results.csv without the row	Deleted
Unit tests (test_app.py)	Validation, ranking, statistics and verdicts	4 of 4 passed
All pages and CSV downloads	HTTP 200; unknown athlete redirects	All returned 200; redirect 302

The tests were run programmatically on a copy of the data files using the submitted source code, and the web pages were exercised with Flask's test client and a headless browser. They should also be tried manually on the computer used for the presentation.

8. Limitations and Future Scope

Limitations: (i) the included observations are synthetic; (ii) conditions such as weather, wind and fatigue are not recorded, although they affect times and distances; (iii) the verdict thresholds are exploratory and eight sessions is a short series; (iv) only three events are configured; and (v) athletes are anonymised but the site has no user accounts or access control, so it should be run locally.

Possible extensions include user login for coaches, a database in place of CSV files, more events and custom ranges, recording of wind and conditions, an edit form for existing results, interactive charts, and forecasting of next-session performance. Any real athlete data should be collected with consent and stored with privacy safeguards.

9. Conclusion

The project demonstrates a complete workflow for exploring athlete performance across sessions: validated input, direction-aware ranking, CSV record management, an object-oriented Athlete class, NumPy statistics, Pandas event reports and Matplotlib bar and time-series charts, all delivered as a Flask web application. On the included data, only one third of athlete-event series show consistent improvement, and the second-ranked 100 m athlete owes that position to a single session. The central conclusion is that rankings and first-to-last improvement figures should always be read alongside a consistency check, which this application provides automatically.

10. References

Python Software Foundation. Python 3 Documentation. https://docs.python.org/3/

Pallets Projects. Flask Documentation. https://flask.palletsprojects.com/

NumPy Developers. NumPy Documentation. https://numpy.org/doc/

Pandas Development Team. pandas Documentation. https://pandas.pydata.org/docs/

Matplotlib Development Team. Matplotlib Documentation. https://matplotlib.org/stable/

Python Software Foundation. csv — CSV File Reading and Writing. https://docs.python.org/3/library/csv.html

11. Appendix: CSV Format and Execution

Required results CSV headers (exact spelling): athlete_id, event, session, date, result.

athlete_id	event	session	date	result
A01	100m	1	2026-08-03	12.46
A01	100m	2	2026-08-10	12.31
A01	100m	3	2026-08-17	12.23

athletes.csv uses the headers athlete_id, squad (for example A01, Senior). To run the project, install the required packages with *pip install -r requirements.txt*, start the server with *python app.py*, and open http://127.0.0.1:5000 in a browser. Run *python test_app.py* to execute the tests and *python seed_data.py* to regenerate the sample data. Use the Records page to import a CSV file or download all results.